The 5 Retrieval Algorithms

A visual guide · ISBA 2411 · Week 4, Lecture 7 · The Retrieval Hackathon

Retrieval means ranking a pile of documents so the right one lands on top. Your five tools fall into three families — one page each follows, with a diagram, how it works, its history, real use cases, and where to read more.

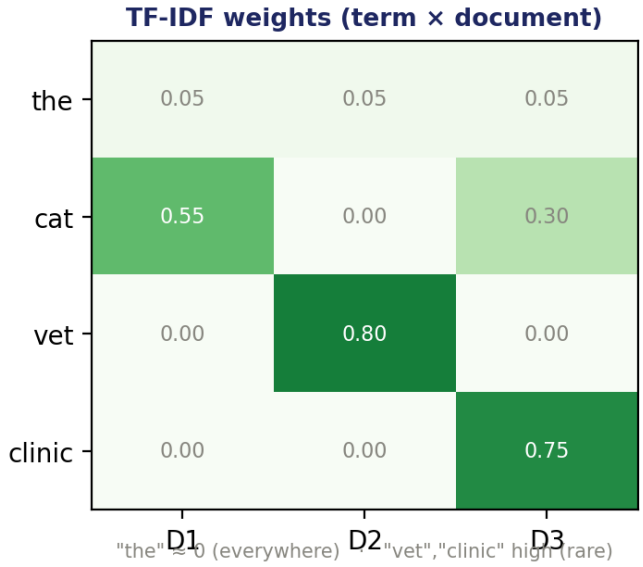
Family	Algorithms	Idea
Keyword (sparse)	1. TF-IDF · 2. BM25	match the words — fast, exact, blind to synonyms
Semantic (dense)	3. Embeddings (bi-encoder)	match the meaning — understands paraphrases
Combine & refine	4. Hybrid · 5. Cross-encoder re-rank	fuse both, then re-score the top few precisely

How they build on each other: start with a keyword **baseline** (TF-IDF/BM25); add a **semantic** model to catch paraphrases; **fuse** them for the best of both; then **re-rank** the top results for a final accuracy boost. More power usually costs more compute — that trade-off is the heart of the hackathon.

1. TF-IDF

Keyword (sparse) · no training

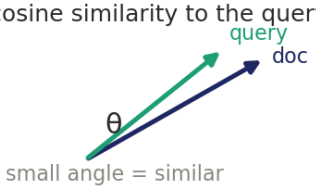
Weight each word by how rare it is, then match.



$$\text{weight}(t,d) = TF(t,d) \times \log\left(N / DF(t)\right)$$

TF = times t appears in d
IDF = rarer across corpus → bigger

Each document is a vector of weights.
Rank by cosine similarity to the query.



How it works

Every document becomes a vector over the vocabulary. A word's weight is its **term frequency** (how often it appears in that document) times its **inverse document frequency** (how rare it is across the whole corpus) — so common words like "the" count for almost nothing while rare, distinctive words dominate. Rank documents by the cosine similarity of their vectors to the query's vector.

History

Term-frequency weighting traces to Hans Peter Luhn (1950s). **Karen Spärck Jones** introduced inverse document frequency in 1972 — the idea that rarer words are more informative. A cornerstone of information retrieval for 50+ years.

Use cases

NLP: search baselines, keyword extraction, document clustering, features for text classification (spam, sentiment), plagiarism detection.
Beyond: matching resumes to jobs, feature-overlap recommendations, log/anomaly analysis.

Strengths fast, interpretable, no model to train.

Watch-outs blind to synonyms and word order — misses paraphrases.

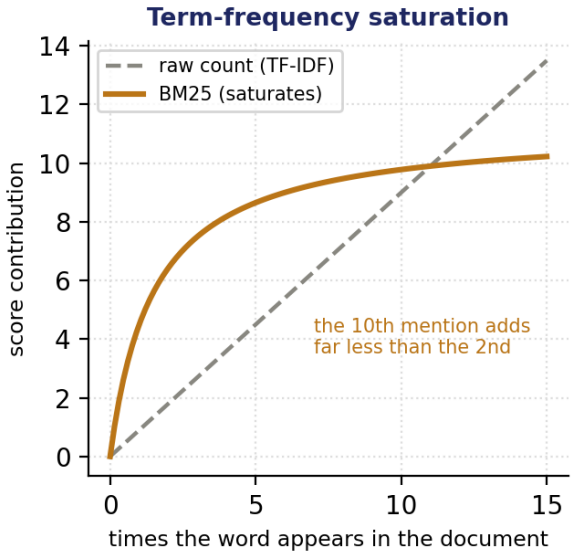
Resources

Jurafsky & Martin, SLP ch. 6 · https://web.stanford.edu/~jurafsky/slp3/6.pdf
Spärck Jones (1972), the IDF paper · https://en.wikipedia.org/wiki/Tf%E2%80%93idf
scikit-learn TfidfVectorizer · https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

2. BM25

Keyword (sparse) · probabilistic

TF-IDF, but smarter about repetition and document length.



BM25 = a smarter keyword score

IDF(t) × TF-saturation(k1) × length-norm(b)

IDF	rare words count more (like TF-IDF)
k1	caps repeated-word inflation
b	stops long docs from winning by size

Still keyword-based — needs shared words.

How it works

An improved keyword score built on the probabilistic relevance framework. Like TF-IDF it rewards rare words, but adds two fixes: **term-frequency saturation** (parameter k1) so the 10th occurrence of a word adds far less than the 2nd, and **length normalization** (parameter b) so long documents don't win just for containing more words. It is the default ranking function in most search engines.

History

Okapi BM25 was developed at City, University of London in the 1980s–90s by **Stephen Robertson, Karen Spärck Jones** and colleagues (the Okapi system). "BM" = Best Matching. Still the default ranker in Lucene, Elasticsearch and Solr.

Use cases

NLP: the workhorse first-stage ranker in search engines, question answering, and RAG pipelines.
Beyond: e-commerce product search, enterprise, legal and medical document search.

Strengths strong, fast, tunable (k1, b); a hard baseline to beat.

Watch-outs still lexical — a paraphrase with no shared words scores zero.

Resources

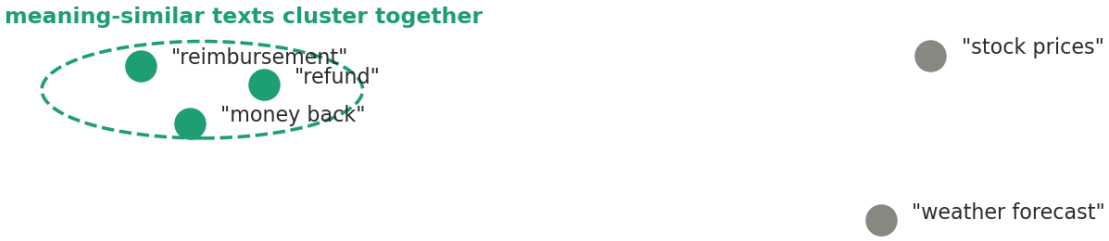
Robertson & Zaragoza (2009), "BM25 and Beyond" · https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf
Okapi BM25 — overview · https://en.wikipedia.org/wiki/Okapi_BM25
rank_bm25 (the library you used) · https://github.com/dorianbrown/rank_bm25

3. Dense embeddings (bi-encoder)

Semantic (neural)

Turn text into vectors where meaning equals closeness.

Text → one vector (encode each passage once)



Search = encode the query, rank passages by cosine similarity in this space

How it works

A neural network (usually a Transformer) maps each text to a fixed-length vector so that meaning-similar texts land near each other in space — even with no shared words. A **bi-encoder** encodes the query and every passage *separately*, so you can embed all passages once (the index) and then search by cosine similarity to the query's vector, fast enough for the whole corpus.

History

Word embeddings — **word2vec** (Mikolov et al., 2013) and GloVe (2014). Sentence embeddings — **Sentence-BERT** (Reimers & Gurevych, 2019). Today's leaders (E5, BGE, GTE) are contrastively trained and compared on the MTEB benchmark.

Use cases

NLP: semantic search, RAG retrieval, paraphrase/duplicate detection, clustering, recommendation.
Beyond: image & multimodal search (CLIP), face recognition, product matching, anomaly detection.

Strengths understands meaning and synonyms; fast to search once indexed.

Watch-outs needs a model (compute); can miss exact terms/numbers; usage details matter (e.g. E5's query:/passage: prefixes).

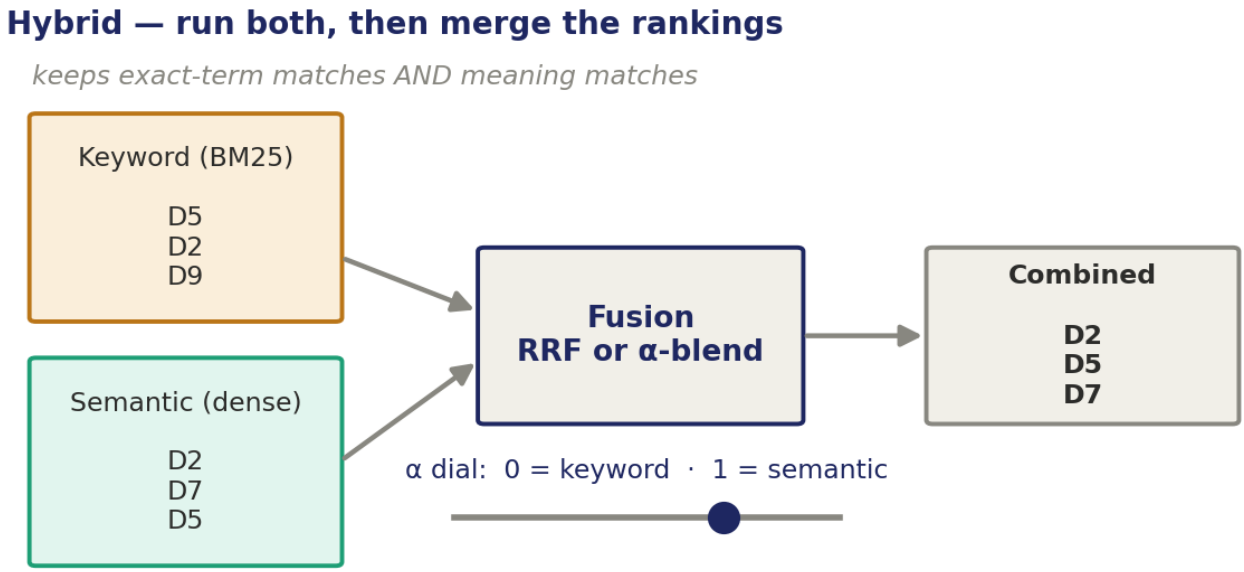
Resources

Reimers & Gurevych (2019), Sentence-BERT · https://arxiv.org/abs/1908.10084
Sentence-Transformers documentation · https://www.sbert.net/
MTEB embedding-model leaderboard · https://huggingface.co/spaces/mteb/leaderboard

4. Hybrid

Combine · lexical + semantic

Run keyword and semantic, then merge — the best of both.



How it works

Run a keyword ranker (BM25/TF-IDF) *and* a dense ranker, then combine their results. Two common recipes: a weighted **score blend** (a dial a`lpha` from 0 = pure keyword to 1 = pure semantic) or **Reciprocal Rank Fusion** (RRF), which merges by each document's rank rather than raw scores. The result keeps keyword's exact-term matches *and* semantic's meaning matches.

History

Data fusion in IR goes back to Fox & Shaw (1994). **Reciprocal Rank Fusion** — Cormack, Clarke & Büttcher (2009) — is the popular, tuning-free recipe. Hybrid search is now built into Elasticsearch, Weaviate, Vespa and Pinecone.

Use cases

NLP: production semantic search and RAG (hybrid retrieval is now common), enterprise search needing both exact terms and meaning.
Beyond: e-commerce (exact SKU + intent), recommendation ensembles, meta-search.

Strengths robust and usually the best overall; captures both failure modes.

Watch-outs two systems to run; a fusion weight (alpha) to tune — do it on dev, not the test.

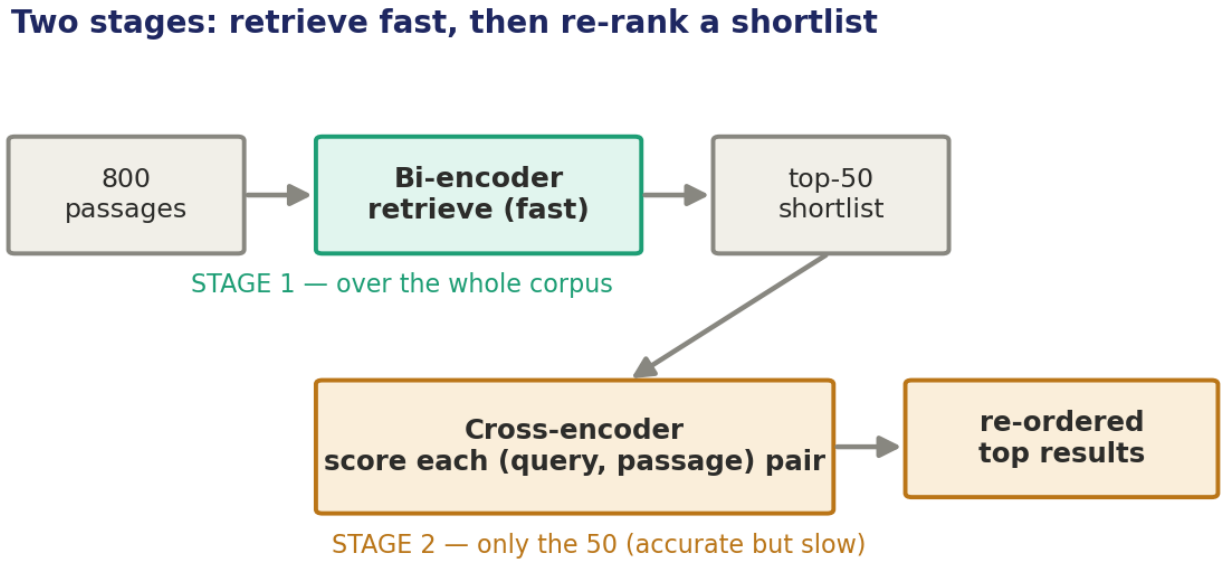
Resources

Cormack et al. (2009), Reciprocal Rank Fusion · https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf
Elasticsearch — hybrid search · https://www.elastic.co/what-is/hybrid-search
Weaviate — hybrid search docs · https://weaviate.io/developers/weaviate/search/hybrid

5. Cross-encoder re-ranking

Refine · neural, two-stage

Read query and passage together for a precise score — on a shortlist.



How it works

A **cross-encoder** feeds the query and a candidate passage *together* through one Transformer that outputs a single relevance score, with full attention between the two. That joint reading is far more accurate than comparing separate vectors — but too slow to run over the whole corpus. So it works in **two stages**: a fast retriever pulls a shortlist (say the top 50), then the cross-encoder re-scores only those pairs and reorders them. Classic "retrieve-and-re-rank."

History

BERT for ranking — **Nogueira & Cho (2019)**, "Passage Re-ranking with BERT," on the MS MARCO benchmark. Cross-encoders became the accuracy leader for reranking; ColBERT (2020) later offered a faster "late-interaction" middle ground.

Use cases

NLP: second-stage reranking in search and RAG, answer selection, semantic-textual-similarity scoring.
Beyond: reranking recommendations, candidate–job matching — any "score this pair" task.

Strengths the highest accuracy of the five — the biggest single quality jump.

Watch-outs slow (scores every pair) — only for a shortlist; a GPU helps a lot.

Resources

Nogueira & Cho (2019), Passage Re-ranking with BERT · https://arxiv.org/abs/1901.04085
Sentence-Transformers — retrieve & re-rank guide · https://www.sbert.net/examples/applications/retrieve_rerank/README.html
MS MARCO benchmark · https://microsoft.github.io/msmarco/